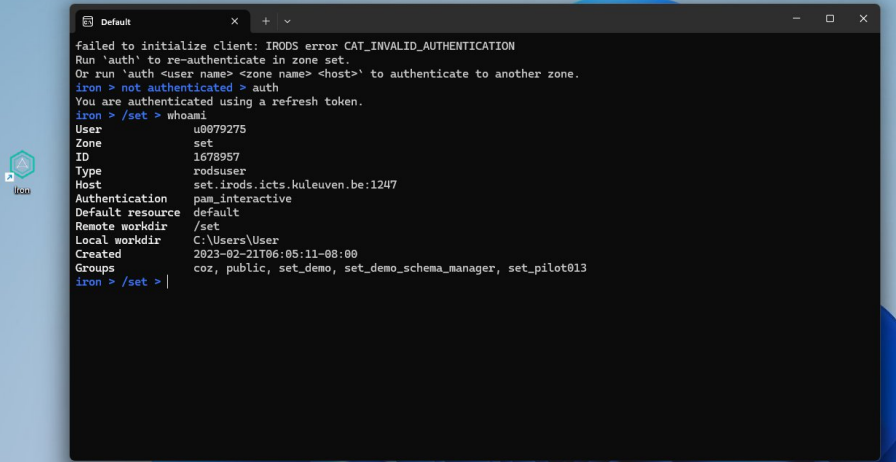


iron: a Go-based iRODS client and library, and a bridge between iRODS and the ownCloud desktop client

iRODS UGM 2026 Barcelona



```

failed to initialize client: IRODS error CAT_INVALID_AUTHENTICATION
Run 'auth' to re-authenticate in zone set.
Or run 'auth <user name> <zone name> <host>' to authenticate to another zone.
iron > not authenticated > auth
You are authenticated using a refresh token.
iron > /set > whoami
User          u8079275
Zone          set
ID            1678957
Type          rodsuser
Host          set.irods.icts.kuleuven.be:1247
Authentication pam_interactive
Default resource default
Remote workdir /set
Local workdir  C:\Users\User
Created       2023-02-21T06:05:11-08:00
Groups        coz, public, set_demo, set_demo_schema_manager, set_pilot013
iron > /set >
  
```

Why another iRODS client?

- Primary goals:
 - a stable *library* allowing integration with existing Go projects (e.g. SFTP)
 - an abstraction of the low level iRODS API into standard Go interfaces
 - functions to open data objects (io.Reader/io.Writer/reopen files)
 - write rich queries just as with PRC
- Evolved to:
 - One portable, fast, dependency-free client
 - Runs on HPC login nodes, laptops, Windows, ...
 - Aimed at (1) OIDC authentication and (2) data transfers

iron in a nutshell

- A pure-Go implementation of the iRODS protocol
 - Talks directly to iRODS server
 - Both XML and native (beta) protocols are supported
- Two things in one: a library and a command-line client
- Has enhanced “walk” and “transfer” functionality
- Ships as a single static binary
 - Linux, macOS, Windows (native + WSL)
- Open source — github.com/kuleuven/iron

The command-line client: Authentication

- Supports native, `pam_password`, `pam_interactive` schemes
 - prepare `irods_environment.json` as usual
 - **`iron auth`** mimicks **`iinit`** and writes `.irodsA`
- Optimized for `pam_interactive` logins:
`iron auth username zone fqdn`
 - writes `irods_environment.json` with `pam_interactive` scheme
 - TLS encryption configured by default
 - Works with a browser on the desktop and headless on HPC (shows URL)

Grant privileges to ManGO (iRODS protocol)

The application ManGO (iRODS protocol) is requesting:

- Basic access to your profile (username, name, email address).
- A long-lived session to act on your behalf without needing re-authentication. You can revoke or extend this session at any time at <https://mango-auth.icts.kuleuven.be>.

Agree and continue to the application

The command-line client: Usage

- Familiar verbs:
`ls, cp, mv, rm, mkdir, stat, tree, checksum, chmod`
- **upload, download** recursively sync a tree
 - transfers are parallelised automatically (chunks & files)
 - files are skipped on modify timestamp or checksum comparison
 - portable alternative for irsync
- shell completion for bash, PowerShell
- iron shell: an interactive session with tab completion (also “GUI”)
- **--admin** flag for rodsadmin accounts

<https://github.com/kuleuven/iron/blob/main/assets/screencast.gif>

iron as a Go library

```
// Recursive walk through the tree, displaying access and metadata
fn := func(path string, info api.Record, err error) error {
    if err != nil {
        return nil
    }

    fmt.Println(path)
    fmt.Printf("%v", info.Access())
    fmt.Printf("%v", info.Metadata())

    return nil
}
```

iron as a Go library

```
results := api.Query(  
    msg.ICAT_COLUMN_D_DATA_ID,  
    msg.ICAT_COLUMN_COLL_ID,  
    msg.ICAT_COLUMN_DATA_SIZE,  
).Where(  
    msg.ICAT_COLUMN_COLL_NAME,  
    fmt.Sprintf(`= '%s'`, coll),  
).Where(  
    msg.ICAT_COLUMN_DATA_NAME,  
    fmt.Sprintf(`= '%s'`, name),  
).Execute(ctx)  
  
defer results.Close()
```

```
for results.Next() {  
    var id, collID, size int64  
  
    err := results.Scan(  
        &id,  
        &collID,  
        &size,  
    )  
    if err != nil {  
        return err  
    }  
  
    fmt.Printf("%d in coll %d has size %d", id, collID, size)  
}
```


iron as a Go library

- We use iron to provision users and groups to iRODS
- Expose iRODS over SFTP — github.com/pkg/sftp
- Custom SFTP ingestion service
 - SFTP server that allows writes only
 - Does not list previously uploaded data
 - Prohibit any read access
- ...

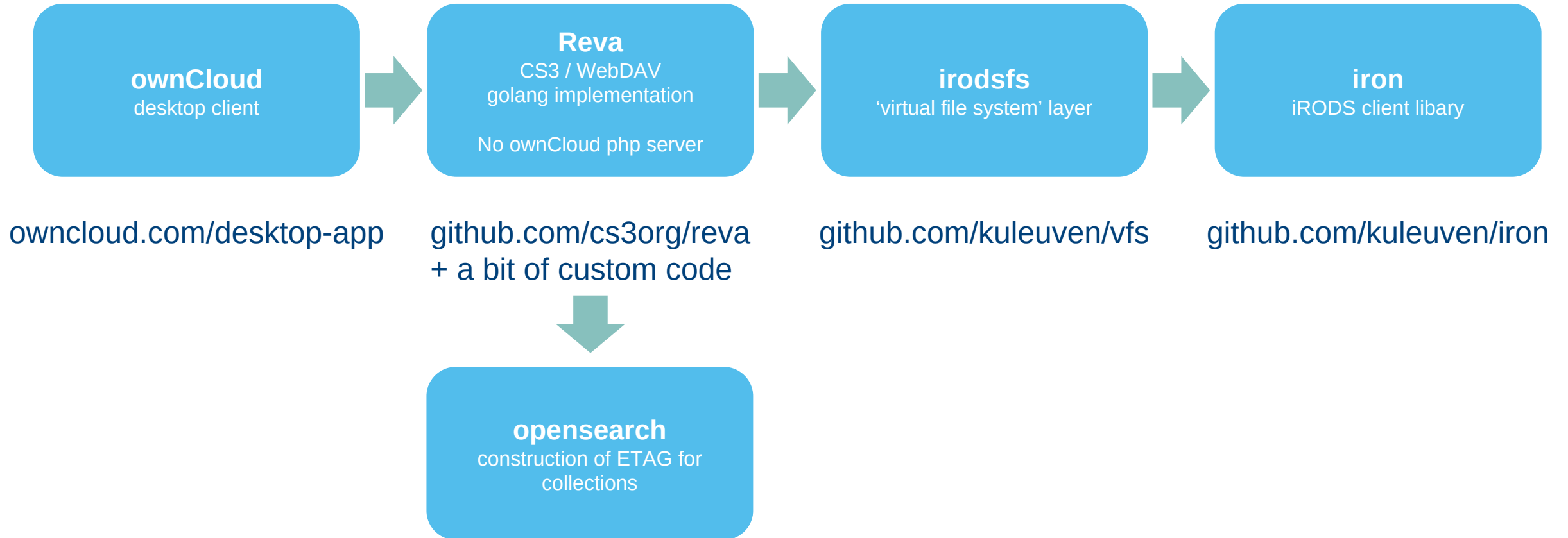
Beyond a client: sync & share

- Researchers already familiar with OneDrive, Dropbox...
- Goal: reach iRODS through a similar desktop sync client
- Self-hosted alternatives: ownCloud and Nextcloud
- CERN developed Reva which serves as web component for ownCloud
 - ownCloud supports OIDC device code flow with token renewals
 - Nextcloud would work for username/password environments
- Our setup
 - Reva server with iRODS as storage backend
 - Standard ownCloud client on user machines
 - With: virtual files, activity view

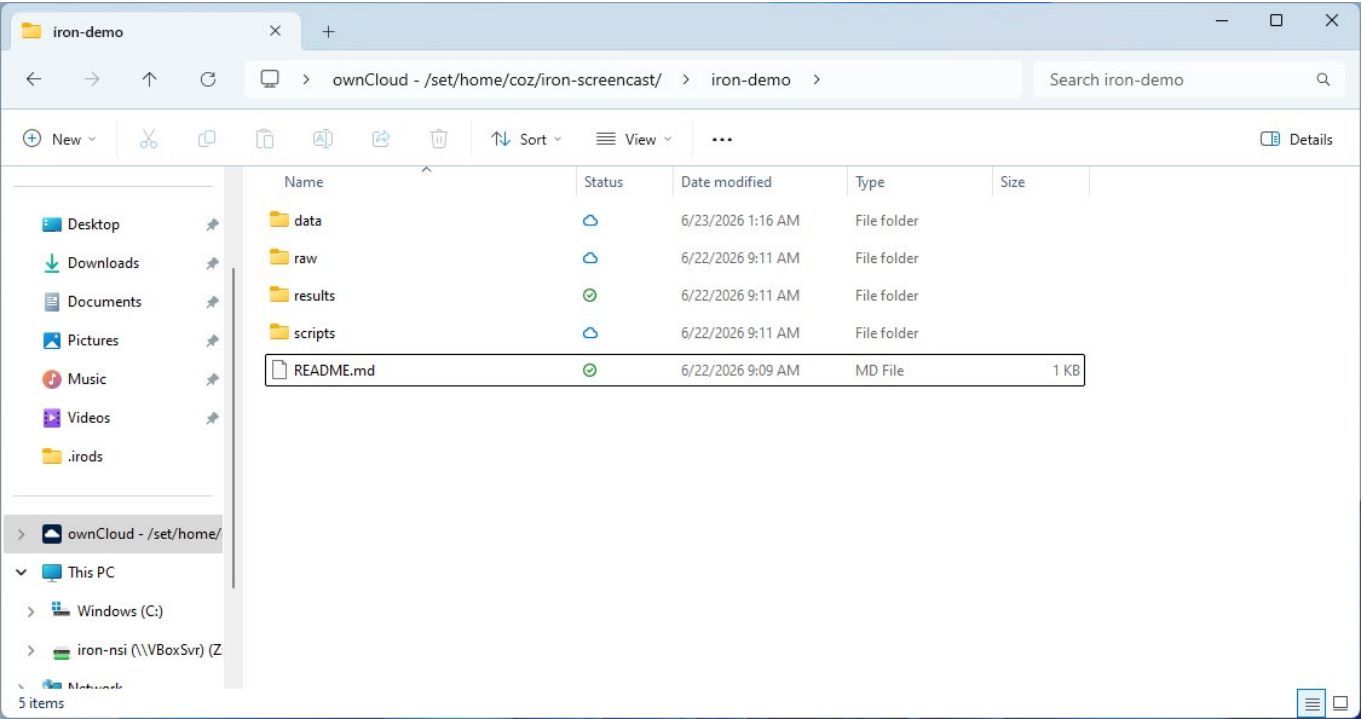
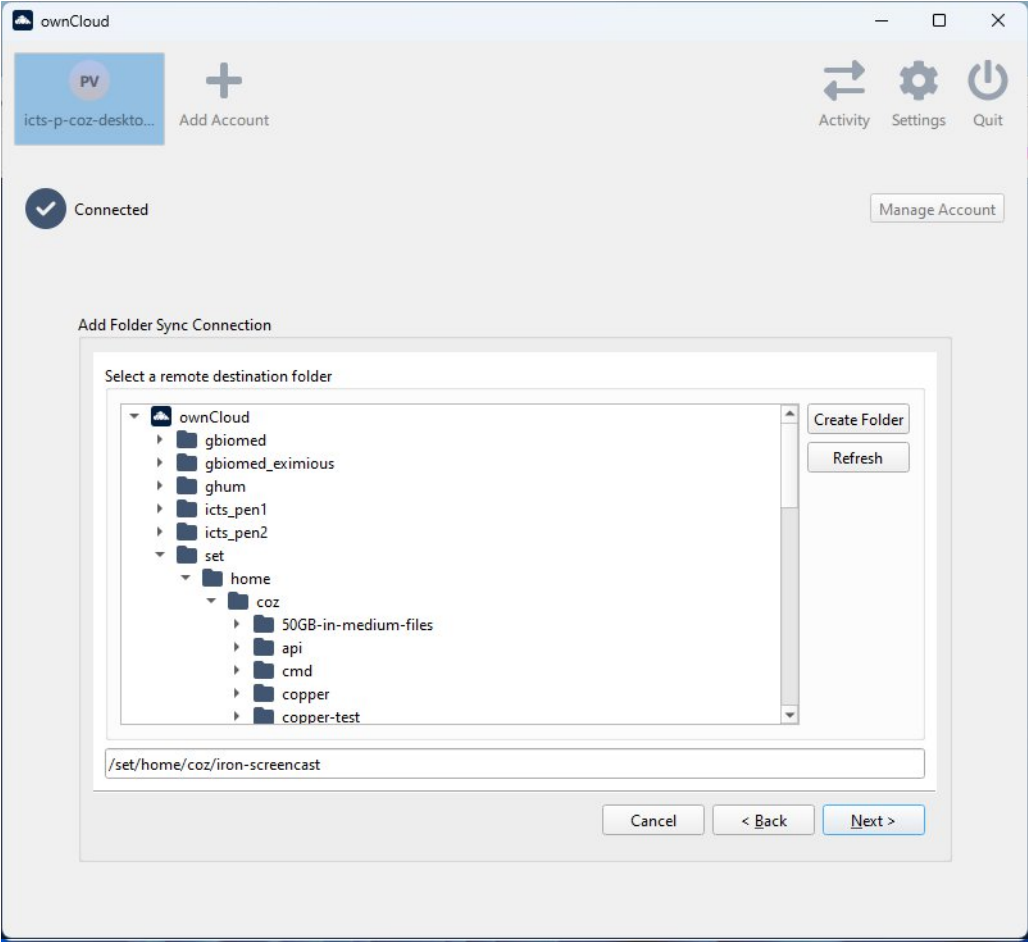
Reva & the CS3 APIs

- Reva is the reference implementation of the CS3 APIs
 - CS3 = Cloud Storage Services for Collaboration
 - Customized webdav for syncing
 - Protocols for third-party copy mechanisms
- Powers ownCloud Infinite Scale (oCIS) and CERNBox
- Decouples the sync & share frontend from storage backends
 - backends plug in as storage providers
 - supporting iRODS = writing storage provider

iron as a Reva storage provider



Demo



Deployment & next steps

- CLI rolled out for VSC Tier-1 Data / KU Leuven ManGO
 - Local IT admins very happy for transfers
- OwnCloud desktop sync client in pilot phase, public rollout to follow
- Next steps:
 - Possibility to build GUI for people still scared of terminals
 - Support for enriched metadata operations
 - Improve ETAGs in Reva / Make custom part open source

The people

